

# Serialize & Transform

Binary Trees, Serialization, Tree Transformations · Mentor-Led Lab  
(Student Edition)

Developed by TalenciaGlobal

**Difficulty** ★★★★★ Advanced

## Problem

Serialize a binary tree to a preorder string with explicit null markers, then transform it: mirror the tree (swap every node's children) and serialize the result.

You are given starter code with two functions. `serialize` is almost complete but has a bug, and `invert` is an empty stub. Fix the bug and implement the stub.

## What to Compute

- Serialized: the preorder serialization of the tree, writing N for every missing child.
- Inverted: the preorder serialization of the mirrored tree, where every node's left and right children are swapped.

## Input / Output Format

**Input:** One line: the tree in level order (values and N).

**Output:** Two lines: Serialized and Inverted.

## Examples

### Example 1

**Input:** tree = 1 2 3 N 4 5 N

**Output:**

Serialized: 1 2 N 4 N N 3 5 N N N

Inverted: 1 3 N 5 N N 2 4 N N N

**Explanation:** Preorder with nulls gives the serialization; mirroring swaps children at every node, so the serialization of the mirror reverses the left/right structure.

### Example 2

**Input:** tree = 1 2 3

**Output:**

Serialized: 1 2 N N 3 N N

Inverted: 1 3 N N 2 N N

**Explanation:** Mirroring swaps the children of the root, so 2 and 3 trade places in the inverted serialization.

## Constraints

---

- Serialize in preorder: emit the value, then the left subtree, then the right subtree.
- Write N for every missing child so the structure is fully captured.
- Inverting swaps each node's left and right child.

## Your Task

---

- Task 1 - Fix the bug. serialize is flagged with a comment, but the bug's location is not given. Find the single bug so the preorder order is value, then LEFT subtree, then RIGHT subtree.
- Task 2 - Implement invert from scratch: mirror the tree by swapping every node's left and right children, and return the new root.

## Starter Code

---

*Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.*

### Python

```
import sys
from collections import deque

class Node:
    def __init__(self, v):
        self.v = v
        self.left = None
        self.right = None

def build(tokens):
    if not tokens or tokens[0] == "N":
        return None
    root = Node(int(tokens[0]))
    q = deque([root])
    i = 1
    while q and i < len(tokens):
        node = q.popleft()
        if i < len(tokens):
            if tokens[i] != "N":
                node.left = Node(int(tokens[i]))
                q.append(node.left)
            i += 1
        if i < len(tokens):
            if tokens[i] != "N":
```

```

        node.right = Node(int(tokens[i]))
        q.append(node.right)
        i += 1
    return root

# BUG: this function contains a single bug. A correct preorder serialization must
# emit each node's value, then its left subtree, then its right subtree.
def serialize(node, out):
    if node is None:
        out.append("N")
        return
    out.append(str(node.v))
    serialize(node.right, out)
    serialize(node.left, out)

def serialize_str(node):
    out = []
    serialize(node, out)
    return " ".join(out)

# TODO: implement this. It must mirror the tree, swapping every node's left and
# right children, and return the new root. Currently a stub that changes nothing.
def invert(node):
    return node

def main():
    tokens = sys.stdin.read().split()
    root = build(tokens)
    serial = serialize_str(root)
    inverted = serialize_str(invert(root))
    print("Serialized: " + serial)
    print("Inverted: " + inverted)

main()

```

## C++

```

// Advanced Tree Lab 6 - Serialize & Transform (STARTER)
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <sstream>
#include <algorithm>
#include <set>
using namespace std;

struct Node {
    long long v;
    Node* left;
    Node* right;
    Node(long long x) : v(x), left(nullptr), right(nullptr) {}
};

Node* build(const vector<string>& t) {
    if (t.empty() || t[0] == "N") {

```

```

    return nullptr;
}
Node* root = new Node(stoll(t[0]));
queue<Node*> q;
q.push(root);
size_t i = 1;
while (!q.empty() && i < t.size()) {
    Node* node = q.front();
    q.pop();
    if (i < t.size()) {
        if (t[i] != "N") {
            node->left = new Node(stoll(t[i]));
            q.push(node->left);
        }
        i++;
    }
    if (i < t.size()) {
        if (t[i] != "N") {
            node->right = new Node(stoll(t[i]));
            q.push(node->right);
        }
        i++;
    }
}
return root;
}

// BUG: this function contains a single bug. A correct preorder serialization must
// emit each node's value, then its left subtree, then its right subtree.
void serialize(Node* node, vector<string>& out) {
    if (node == nullptr) {
        out.push_back("N");
        return;
    }
    out.push_back(to_string(node->v));
    serialize(node->right, out);
    serialize(node->left, out);
}

string serializeStr(Node* node) {
    vector<string> out;
    serialize(node, out);
    string s;
    for (size_t i = 0; i < out.size(); i++) {
        if (i) {
            s += " ";
        }
        s += out[i];
    }
    return s;
}

// TODO: implement this. It must mirror the tree, swapping every node's left and
// right children, and return the new root. Currently a stub that changes nothing.
Node* invert(Node* node) {
    return node;
}

```

```

int main() {
    vector<string> tokens;
    string s;
    while (cin >> s) {
        tokens.push_back(s);
    }
    Node* root = build(tokens);
    string serial = serializeStr(root);
    string inverted = serializeStr(invert(root));
    cout << "Serialized: " << serial << "\n";
    cout << "Inverted: " << inverted << "\n";
    return 0;
}

```

## Java

// Advanced Tree Lab 6 - Serialize & Transform (STARTER)

```
import java.util.*;
```

```

public class Main {
    static class Node {
        long v;
        Node left, right;
        Node(long x) {
            v = x;
        }
    }

    static Node build(String[] t) {
        if (t.length == 0 || t[0].equals("N")) {
            return null;
        }
        Node root = new Node(Long.parseLong(t[0]));
        Queue<Node> q = new LinkedList<>();
        q.add(root);
        int i = 1;
        while (!q.isEmpty() && i < t.length) {
            Node node = q.poll();
            if (i < t.length) {
                if (!t[i].equals("N")) {
                    node.left = new Node(Long.parseLong(t[i]));
                    q.add(node.left);
                }
                i++;
            }
            if (i < t.length) {
                if (!t[i].equals("N")) {
                    node.right = new Node(Long.parseLong(t[i]));
                    q.add(node.right);
                }
                i++;
            }
        }
        return root;
    }
}

```

// BUG: this function contains a single bug. A correct preorder serialization must

```

// emit each node's value, then its left subtree, then its right subtree.
static void serialize(Node node, List<String> out) {
    if (node == null) {
        out.add("N");
        return;
    }
    out.add(Long.toString(node.v));
    serialize(node.right, out);
    serialize(node.left, out);
}

static String serializeStr(Node node) {
    List<String> out = new ArrayList<>();
    serialize(node, out);
    return String.join(" ", out);
}

// TODO: implement this. It must mirror the tree, swapping every node's left and
// right children, and return the new root. Currently a stub that changes nothing.
static Node invert(Node node) {
    return node;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    List<String> tokens = new ArrayList<>();
    while (sc.hasNext()) {
        tokens.add(sc.next());
    }
    Node root = build(tokens.toArray(new String[0]));
    String serial = serializeStr(root);
    String inverted = serializeStr(invert(root));
    System.out.println("Serialized: " + serial);
    System.out.println("Inverted: " + inverted);
}
}

```

## Sample Run

### Sample Input

```
1 2 3 N 4 5 N
```

### Sample Output

```
Serialized: 1 2 N 4 N N 3 5 N N N
Inverted: 1 3 N 5 N N 2 4 N N N
```